

Take-home exercise — AI Engineer (Customer-Support AI)

Thanks for your interest. This mirrors what you'd actually work on: a customer-support AI that answers questions **grounded** in approved content and real account data — safely, and at scale.

Read this first — it's the whole point. This is **not a spec to implement**. There is **no reference solution**. The materials are deliberately **incomplete, occasionally ambiguous, and contain things the bot should refuse to do**. The exercise is to **make the decisions yourself** — what the bot should do in each situation, what to build, what to leave out — and to **justify them**. A flawless build with decisions you can't defend is a fail; a smaller build with sharp, well-reasoned decisions is a pass.

On AI tools: use them if you like — we do. But the *decisions must be yours*. We'll sit with you and walk through every meaningful choice, ask "why this and not that," and extend the solution together, live. The code is the easy part to generate; the judgment is what we're hiring for and what we'll probe.

Don't gold-plate. We're not measuring stamina or breadth — stop once you've shown your thinking.

Part 1 — A grounded support answerer

Given a **customer id** and a **question**, your program returns a short, friendly reply. Everything is in `materials/`: `knowledge.md` (approved help-center answers), `customers.json` (mock account data, with a `safe` vs `restricted` split), and `questions.txt` (questions to run).

This is a decision exercise, not a checklist. Run the questions and you'll hit situations the materials *don't* cleanly resolve. For each, **you decide the right behaviour and record why**. For example, you'll have to decide:

- What should the bot do when the answer **isn't in the approved content at all**? (Some questions have no approved answer. Some are out of scope entirely.)
- What should it do when it **can see a problem in the account data** but has **no approved policy** for it?
- What should happen when a customer asks for information that's in the data but **shouldn't be shared**?

- How does it tell "I can answer this confidently" from "a human should take this" — and what does it do in each case?
- The approved content and the account data are small and imperfect. Where they're **ambiguous, missing, or could be read two ways**, that's a decision for you — make it, and note it.

Build *enough* to demonstrate the decisions you made — the retrieval, the grounding, the safety, the escalation are all your design. Simple is fine if you can justify it. **Tests** should demonstrate the behaviours you chose (especially the ones where you decided the bot must *not* do something).

You may use any language and any LLM — or **stub the LLM** with a deterministic stand-in if you'd rather focus on the decisions and the architecture. We are not grading the model; we're grading your design.

Part 2 — A scalable, governed data layer (design)

Today this bot gets a customer's account data by **scraping other microservices' logs** out of a log store — brittle and ungoverned. **Design something better.** A short write-up (+ a diagram) describing how the bot should fetch **live customer data** (balance, card status, KYC, transactions) from **several different owning services**, plus **behavioural analytics from a product-analytics tool**.

There is no correct architecture here — we want **your** reasoning. At minimum, take a clear position on: who owns what data and how it's exposed; how you guarantee the bot never reads the wrong customer's data or a field it shouldn't see; what happens when a service is slow or down mid-answer; data residency for financial data; how a new data signal gets added later; and how it scales as services and AI use-cases multiply. **State your trade-offs and what you'd deliberately defer.**

What to submit

1. **The repo** — code + tests + a `README` (how to run it).
2. `DECISIONS.md` — the heart of your submission. For each meaningful choice (how you ground answers, how you handle the no-approved-answer and the don't-share cases, how you decided answer-vs-escalate, what you cut and why, and how you resolved any ambiguity in the materials): the decision, the alternatives you considered, and why you chose. Be honest about what you're unsure about.
3. **The design note** (Part 2) — PDF or markdown.

We'll read all three, then book a call to walk through your decisions and build on the solution together. If anything is ambiguous — **that's intentional**. State your assumption, decide, and move on. How you handle the ambiguity *is* the exercise.